# VOLUME 01 / 08

# RTL FOUNDATIONS

## Set up the tools and learn to think in clock edges

Build a working environment, understand binary values and write a self-checking clocked RTL experiment.

FIG 02 / REGISTER BOUNDARY

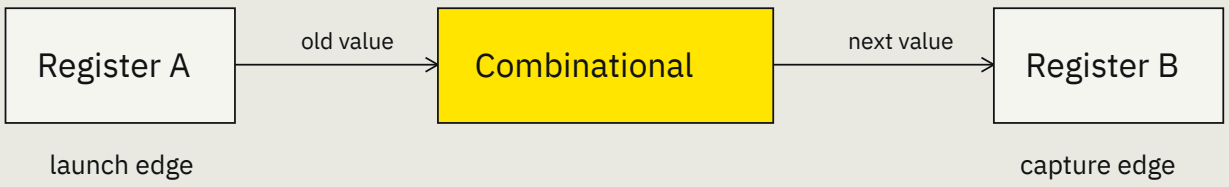



Illustration: combinational logic settles between active edges.

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

# What you will build

You will reproduce the project tools and run a small two-register experiment before editing the CPU. The output is a passing simulation and a written timing prediction.

### Before you start

You need a terminal, Git and Docker. No FPGA or RISC-V compiler is required.

### Files and outputs

Dockerfile, Makefile, docs/textbook/series/labs/rtl_basics.sv. Generated files stay in build/learning-series/.

### Build order

1. Create a learning branch and reproduce the reference.
2. Learn bit widths, signed interpretation and clocked updates.
3. Run the isolated RTL lab and deliberately change its prediction.
4. Write the first experiment entry.

---

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

# A bit pattern needs an interpretation

A **bit** holds 0 or 1. A **byte** contains eight bits. A 32-bit word contains four bytes. Binary positions carry powers of two, with the least significant bit at position zero.

Hexadecimal uses one digit per four bits. This makes a 32-bit instruction readable as eight hex digits. The prefix `0x` means hexadecimal.

> **WORKED EXAMPLE**
>
> $0x2D = 0010\ 1101_2 = 32 + 8 + 4 + 1 = 45.$

Bitwise AND keeps positions where both inputs have a 1. OR keeps positions where either input has a 1. XOR keeps positions that differ. These operations act on each bit separately.

A **multiplexer**, or mux, selects one input value. Forwarding uses muxes to choose between register values and newer pipeline results.

FIG 01 / UNSIGNED BYTE

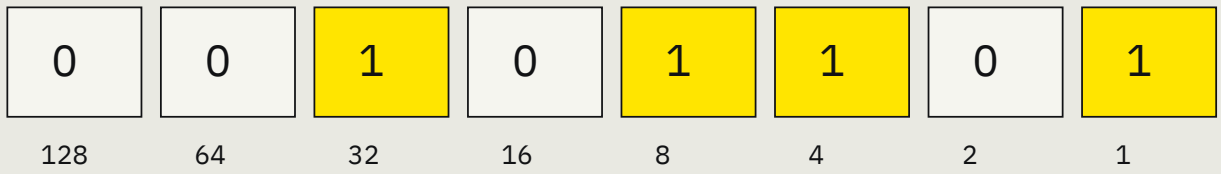



Illustration: each set bit contributes its place value.

# Signed arithmetic reuses the same bits

An unsigned byte spans 0 through 255. A signed two's-complement byte spans −128 through 127. For an n-bit value with its top bit set, subtract $2^n$ from its unsigned value.

> **WORKED EXAMPLE**
>
> `0xFF` is 255 unsigned, but `255 - 256 = -1` signed. `0x80` is 128 unsigned, but −128 signed.

**Sign extension** copies the sign bit into new upper positions. Extending signed `0x80` to 32 bits gives `0xFFFFFF80`. Zero extension gives `0x00000080`.

A logical right shift inserts zeros. An arithmetic right shift copies the sign bit. RV32I uses SRL for the first and SRA for the second. Signed comparisons interpret the sign bit. Unsigned comparisons treat every bit as magnitude.

Integer addition wraps modulo $2^{32}$ in this core. `0xFFFFFFFF + 1` gives zero. Ordinary ADD does not raise an overflow trap.

| Bits | Unsigned 8-bit | Signed 8-bit | 32-bit sign extension |
|---|---|---|---|
| 01111111 | 127 | 127 | 0x0000007F |
| 10000000 | 128 | −128 | 0xFFFFFF80 |
| 11111111 | 255 | −1 | 0xFFFFFFFF |

# A clock edge separates old state from new state

**Combinational logic** computes outputs from current inputs. An adder, comparator or mux is combinational. A **register** stores bits across time. It updates at an active clock edge.

Between edges, signals travel through logic and wires. Before the next edge, the destination register needs a stable input. A path that takes too long limits clock frequency.

> **WORKED EXAMPLE**
>
> With a 20 ns clock period, the nominal frequency is `1 / 20 ns = 50 MHz`. The useful combinational budget is smaller because registers require setup time and clocks have uncertainty.

A pipeline puts registers between work regions. It overlaps different instructions across those regions. It does not make one addition logically need fewer steps.

The actual core uses a synchronous, active-high reset. Reset takes effect on a rising edge. It clears pipeline valid bits and sets the program counter to zero.

FIG 02 / REGISTER BOUNDARY

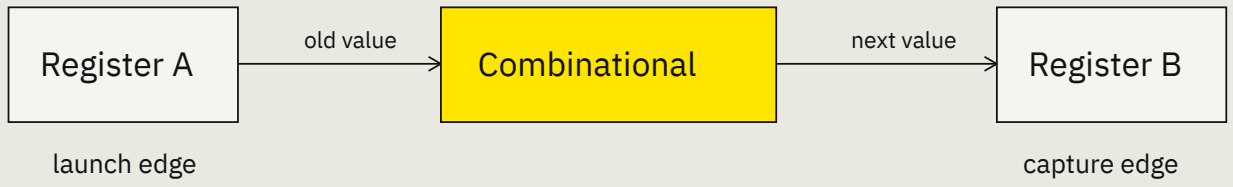



Illustration: combinational logic settles between active edges.

# RTL describes concurrent hardware

**Register-transfer level**, or RTL, describes stored state and the logic between registers. Verilog source order is not a CPU instruction schedule. Separate hardware blocks operate concurrently.

```
always @(posedge clk) begin
    if (reset) begin a ⩽ 0; b ⩽ 0; end
    else begin a ⩽ input_value; b ⩽ a; end
end
```

Nonblocking assignments (⩽) schedule state changes after the block evaluates. Here, b receives the old a. That behavior creates a pipeline boundary.

```
always @* begin
    result = 0;
    case (select)
        0: result = a + b;
        1: result = a ^ b;
        default: result = 0;
    endcase
end
```

Blocking assignments (=) are appropriate for this combinational calculation. Give outputs defaults so every path defines a value. A missing assignment can infer a latch, an unintended storage element.

# Step 1: establish a working baseline

Run these commands from the repository root. Keep the full terminal output in your notes. The image installs Icarus, Verilator, Yosys and nextpnr for later volumes.

```
git switch -c learning/build-core
docker build -t rv32-tools .
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD:/work" rv32-tools make test lint
```

A passing run establishes that your environment can execute the reference. It does not establish that a new edit is correct. Commit or preserve the baseline revision before changing the RTL.

If the branch already exists, use `git switch learning/build-core`. If Docker cannot connect, solve the installation or daemon problem before changing processor code.

Write down what the testbench supplies: a clock, reset, separate instruction/data memories and an EBREAK stopping convention. The core itself does not contain those memories.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 1A. Which files would you send to synthesis? Which files exist only to simulate and check behavior?

# Step 2: read widths and operators

In `reg [31:0] value`, the index range includes 32 bits. In `value[7:0]`, the slice includes eight bits. A 5-bit register number can name 32 registers.

```
wire [31:0] extended = {{24{byte_value[7]}}, byte_value};
wire [31:0] logical_shift = value >> amount;
wire signed [31:0] arithmetic_shift =
    $signed(value) >>> amount;
```

The first expression repeats the sign bit 24 times, then appends the byte. The signed cast in the arithmetic shift matters. Declaring some other signal signed does not make every intermediate expression signed.

A `reg` declaration is a Verilog variable kind. Its driving process determines whether synthesis infers storage. Learn the operation and width of every expression before assuming it maps to a specific circuit.

Check small examples: 0x80 extends to 0xFFFFFF80, and arithmetic shifting −8 by two yields −2. Logical shifting the same 32-bit pattern yields 0x3FFFFFFE.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 1B. Why does a 16-bit signed DOT4 sum fail even though each input lane has only eight bits?

# Step 3: run a two-register experiment

The supplied lab is a complete standalone module and testbench. It uses the same nonblocking edge discipline as the core, without any instruction decoding.

```
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD:/work" rv32-tools make learning-labs
```

The lab first captures 5 into a while b captures old a=0. At the next edge it captures 9 into a while b captures old a=5. Its assertions check those values.

Open `docs/textbook/series/labs/rtl_basics.sv`. Predict the effect of replacing `b <= a` with a combinational assignment. Explain the change before running a modified experiment.

Do not use source statement order to invent sequential behavior inside a combinational block. Do not replace every assignment with one operator mechanically. Choose assignments based on the intended hardware and scheduling.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 1C. Before the second active edge, a=5, b=0 and input=9. Give both new register values.

# Step 4: write synthesizable next-state logic

Start a combinational block with defaults for every output it drives. Then override them for selected cases. An output that retains a previous value on an uncovered path can infer a latch.

```
always @* begin
    result = 0;
    write_enable = 0;
    case (operation)
        2'b00: begin result = a + b; write_enable = 1; end
        2'b01: begin result = a ^ b; write_enable = 1; end
        default: begin result = 0; write_enable = 0; end
    endcase
end
```

This is an isolated pattern, not a new core module. In the actual core, find the defaults for `result`, `wr`, `ld`, `st`, `trap` and `taken`. Follow every override.

Use `make lint` after changes. A clean lint result catches many mistakes but does not prove instruction behavior.

# Debugging the first simulation

### Unknown values

Check whether reset actually reaches a rising edge, whether memory contents are initialized and whether every combinational result is assigned. Inspect the earliest unknown value rather than the final symptom.

### A value arrives one cycle later

Draw old state and scheduled new state at each edge. The extra cycle may be a real register boundary, not simulator lag.

### The testbench races the core

The teaching labs change inputs on falling edges and inspect state after the rising-edge updates settle. Avoid driving and checking the same edge without understanding simulator scheduling.

### Synthesis rejects testbench code

Delay statements, simulation clocks and file-loading behavior belong to testbenches. Synthesize the documented hardware top, not `tb`.

Checkpoint: explain the difference between a wire value, a stored register, and a valid instruction. Record one failed prediction and what corrected it.

# A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

| | |
|---|---|
| **Question** | Do the two clocked assignments use old values? |
| **Prediction** | b receives the previous a at each edge. |
| **Experiment** | Run rtl_basics_tb, change one expected value, observe failure and restore it. |
| **Evidence** | Old/new state table and the standalone simulation log. |

## What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

# Answers and completion check

**1A.** Synthesize the hardware in `rtl/` using the chosen top. `tests/tb.sv`, the Python checker and the standalone teaching bench belong to simulation.

**1B.** Four products of −128×−128 sum to 65536. Signed 16-bit storage stops at 32767. The full sum needs at least 18 signed bits.

**1C.** a becomes 9 and b becomes 5. Both right-hand sides use the old state.

### Ready for Volume 2

You can build the tools, run the standalone lab, explain nonblocking assignments and calculate signed extensions. Save your notes before moving into instruction decode.

# Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. RISC-V ISA
2. Icarus documentation
3. Verilator guide
4. Project source

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

## Keep the evidence boundary

Teaching labs are separate from the 144-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, external ISA certification, formal proof or matched application speedup is implied.

Continue with Volume 2. Run the complete regression before treating a learning edit as a finished implementation.